

CP372 Assignment 2 – Reliable Data Transfer

June 27th, 2026

Jacob Kachou, Om Thesiya, Jagtar Sandhu

Introduction

The purpose of this assignment was to conduct performance and reliability testing for the Stop-and-Wait and Go-Back-N data transfer protocols running on UDP using the Python programming language. Dummy files ranging in size from 10 KB to 100 MB were created and sent over a localhost connection across five trials at each file's size and packet loss rate. These were averaged to compare the performance of these two protocols with regards to packet loss and error recovery. We created artificial packet drops at the destination for all file sizes, where transfer time, throughput, and number of retransmissions were recorded at simulated packet loss rates of 0%, 10%, 20%, and 30%.

The User Datagram Protocol (UDP) is a fast, connectionless networking protocol, which has no way of ensuring that a packet will be delivered to the receiving application or that a packet will arrive in the same order it was sent. Because UDP lacks mechanisms for reliability by itself, it must have a layer of reliability added on to its operations. In TCP/IP implementations, there are three ways to add reliability on top of UDP to provide complete reliability of packet delivery: sequence numbers, which are used to keep track of and arrange data in the correct order; acknowledgments, which are sent from the receiver back to the sender to confirm what data has been received; and timeouts that will generate retransmission of lost data after waiting for a specified amount of time.

Two reliable transfer models were implemented as part of the assignment: Stop-and-Wait, which sends one packet and waits for an acknowledgement from the receiver before sending the next packet; and Go-Back-N, which uses a sliding window to allow multiple packets to be sent before needing an acknowledgement, and it uses cumulative acknowledgements to control the flow of the packets. Selective Repeat was also implemented as an optional component of the assignment, and this is discussed in detail in Section 3.3.

Packet Structure:

The header of both protocols is the same custom binary header, which consists of four 32 bit unsigned integers, that have been arranged into a fixed 16 byte header with Python's struct.pack method followed by the actual data (the payload) itself. The four fields of the header are as follows:

- 1) Packet Type - Indicates what type of packet this is (FILE_START, DATA, FILE_END and ACK)
- 2) Sequence Number - The unique number that is assigned to each packet in sequence, with the next number being one greater than the previous packet's number
- 3) ACK Flag - A flag set to either a 1 or a 0 indicating if this packet is an acknowledgement
- 4) Data Size - The amount of bytes in the payload, which can be less than or equal to 1024 bytes, and does not include the 16 byte header in that number.

3.1 Stop-and-Wait Design

Timer Design

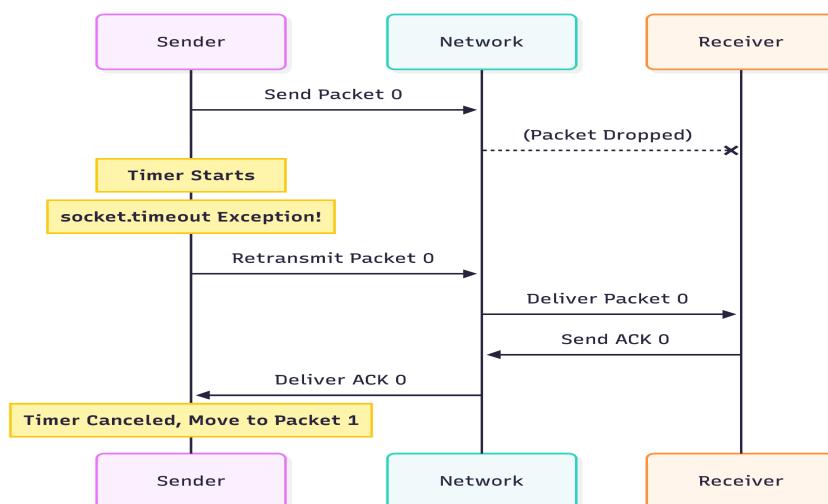
Using Python's `socket.settimeout()` function, the timer waits for an acknowledgment after sending a packet with the `block()` function. If no acknowledgment has been received by the time the timer expires, the socket raises a “timeout” exception and the packet is retransmitted. We can adjust the timeout value which is set by the user. With our tests running only on localhost, there was a round trip delay of only a few microseconds to get back to the sender; therefore, we set a very short timeout period of approximately 10 milliseconds to minimize the delay incurred as a result of an acknowledgment being lost.

ACK Handling

The receiver acknowledges every packet received with an acknowledgment (ACK) of the packet just received and the associated acknowledgment packet contains only the sequence number associated with the just received packet. Once an acknowledgment(ACK) containing the sequence number of the packet currently being transmitted is received by the sender, then the sender will proceed with the transmission of the next packet of data.

Window Implementation

The window size for Stop-and-Wait is that of 1 ($N = 1$). Therefore, at any given time, only one unacknowledged packet can be in existence between the sender and the receiver; this is both a contributing factor to the protocol's simplicity and its inherent slowness.



3.2 Go-Back-N (GBN) Design

Packet Format

Both GBN and Stop-and-Wait protocols employ a standardized 16-byte header called (IIII). By using the same header for both protocols, the application layer remains constant and the receiver is able to distinguish between file setup messages (e.g., FILE START and FILE END) versus actual data.

Timer Design

GBN uses one timer to keep track of when the oldest unacknowledged packet is due for reception, called the base. This timer is not a separate timer for every packet being sent. When the timer expires while waiting to receive Acknowledgements (ACKs), the sender will retransmit all of the packets that were sent from the base (from the base until $(\text{next_seq_num} - 1)$ in order.

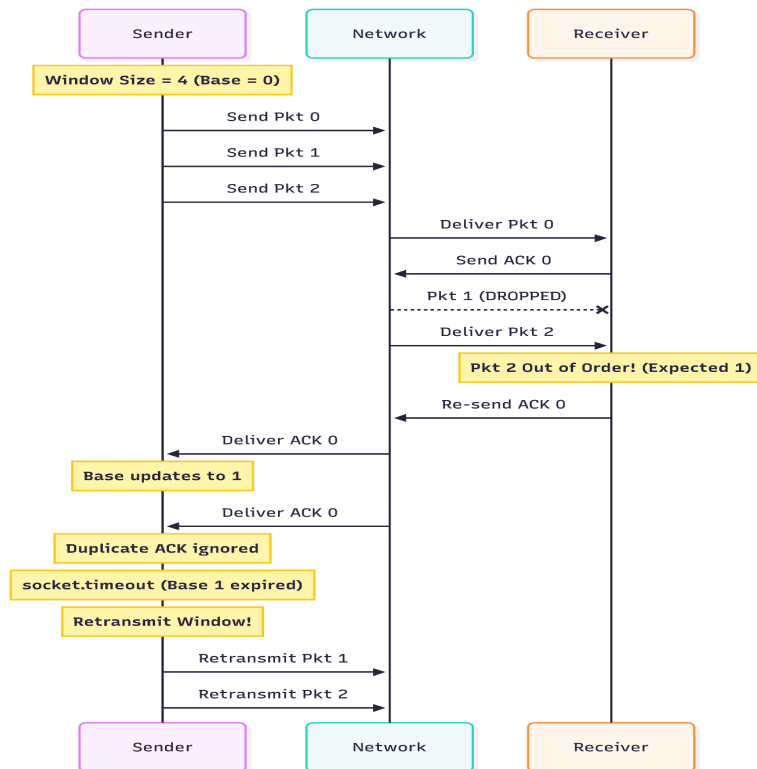
ACK Handling

Cumulative ACKs are used by the Go-Back-N Protocol.

The receiver maintains a variable called `expected_seq`, which is for the next packet that will arrive. If a packet arrives in order, the receiver writes the data out, increments the `expected_seq`, and sends an ACK back to the sender, where the number on the ACK matches the sequence number of the packet just received. If the packet arrives out of order, the receiver drops the packet and sends back an ACK for `expected_seq - 1`, which will only acknowledge the last in-order packet it received. Each time a valid ACK is received at the sender, it will move the base value forward based on the sequence number of the ACK received, and slide the window forward.

Window Implementation

A sender will utilize a sliding window of $N=10$, with two major state variables, `base`, which represents the oldest unacknowledged packet, and `next_seq_num`, which is the next Packet to be sent out. The sender will continue sending packets whenever `next_seq_num` is less than $(\text{base} + \text{window_size})$, filling the window without pausing for ACKs on individual packets sent.



3.3 Selective Repeat Design

Selective Repeat using a single timer was implemented to satisfy our bonus implementation criteria. The packet header, specifically a 16 byte header is used by Selective Repeat as well as FILE START, DATA and FILE END as confirmed handshaking for multiple packet transmission and reception.

The Selective Repeat protocol employs three main differences relative to Go Back N:

- Received data is buffered by the receiver until all associated data (missing) can be delivered(out of order) to the applicable application entity.
- ACK information is provided to identify each unique received packet. The sender will know which packets were received since they will receive ACK information for all other packets except those associated with the missing packets.
- The sender will only retransmit (upon timer expiration) packets that have not yet been acknowledged. The sender will only retransmit the unacknowledged packets for which it is timed out, not the entire window of packets previously transmitted (as is the case with Go Back N).

To satisfy the requirement of utilizing only a single timer instance, the socket timeout acts as a single logical timer associated with the oldest unacknowledged packet sent. As this packet is acknowledged and moves forward in the sequence of packets, this timer moves forward to act upon the next oldest unacknowledged packet sent.

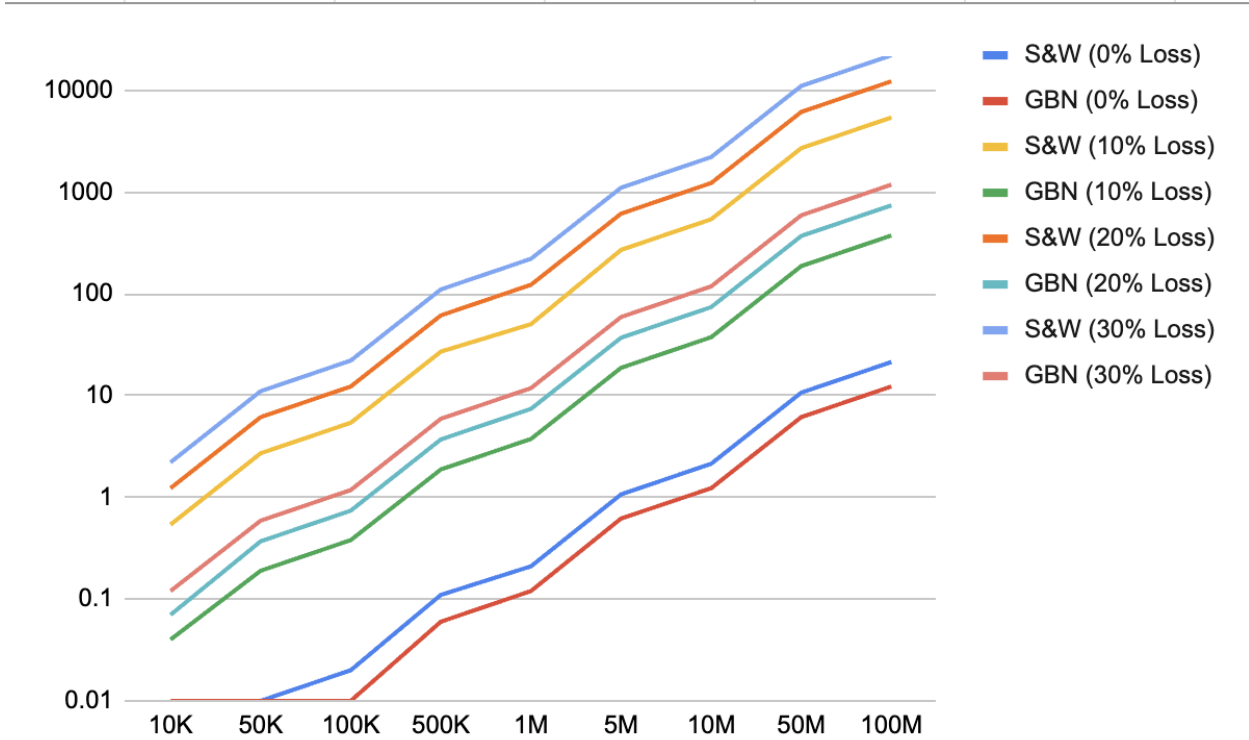
4. Experimental Results

Table 1. Average Stop-and-Wait Transfer Time (s) — avg of 5 trials									
Loss Rate	10K	50K	100K	500K	1M	5M	10M	50M	100M
0%	0.0000	0.0020	0.0100	0.0300	0.0400	0.1180	0.2100	1.0000	1.9100
10%	0.0220	0.0740	0.1460	0.7320	1.3980	7.0020	13.9900	70.8133	146.6167
20%	0.0280	0.1460	0.3240	1.5520	2.9660	24.9500	30.3540	161.4733	322.3267
30%	0.0480	0.2480	0.5180	2.6460	4.8840	26.2640	53.8640	264.0733	516.4867

Table 2. Average Go-Back-N Transfer Time (s) — avg of 5 trials									
Loss Rate	10K	50K	100K	500K	1M	5M	10M	50M	100M
0%	0.0008	0.0016	0.0021	0.0074	0.0138	0.0484	0.0867	0.4209	0.8153
10%	0.0131	0.0709	0.1257	0.6759	1.3915	7.3901	14.4251	72.3582	144.0598
20%	0.0292	0.1816	0.3006	1.5205	3.3289	16.0691	31.6748	160.2606	319.1411
30%	0.0515	0.2467	0.5279	2.8763	5.6335	26.9007	54.3796	272.3948	558.9947

Table (Bonus). Average Selective Repeat (Bonus) Transfer Time (s) — avg of 5 trials									
Loss Rate	10K	50K	100K	500K	1M	5M	10M	50M	100M
0%	0.0000	0.0000	0.0000	0.0100	0.0100	0.0500	0.0920	0.4200	0.8167
10%	0.0080	0.0360	0.0720	0.3440	0.7440	3.7600	7.5000	37.7200	75.8000
20%	0.0200	0.0760	0.1240	0.6100	1.2440	6.3860	12.4160	62.9733	129.7300
30%	0.0160	0.0920	0.1620	0.8260	1.8840	9.0120	18.8780	97.1333	194.2700

Table 4. Comprehensive Performance Metrics (100 MB File, avg of 3 trials)				
Protocol	Loss Rate	Avg Time (s)	Avg Throughput (bytes/s)	Avg Retransmissions
Stop-and-Wait	0%	1.91	54,975,450	0
Stop-and-Wait	10%	146.62	715,536	11,279
Stop-and-Wait	20%	322.33	325,321	25,622
Stop-and-Wait	30%	516.49	203,295	43,845
Go-Back-N	0%	0.82	128,675,111	0
Go-Back-N	10%	144.06	727,878	11,399
Go-Back-N	20%	319.14	328,568	25,572
Go-Back-N	30%	558.99	187,853	44,006
Selective Repeat	0%	0.82	128,195,326	0
Selective Repeat	10%	75.80	1,383,729	11,306
Selective Repeat	20%	129.73	809,393	25,759
Selective Repeat	30%	194.27	539,755	44,040



Analysis

1. No. At zero percent loss for a 100MB file size, Go-Back-N had a finish time of 0.82 seconds and Stop-and-Wait required 1.91 seconds. Therefore Go-Back-N was approximately 2.3 times faster as it had an effective throughput of 129MB/s compared to Stop-and-Wait's effective throughput of 55MB/s. The reason for this difference in speed is because of the pipelining capability of Go-Back-N where Go-Back-N can have up to ten outstanding packets at once as compared to one packet at a time with Stop-and-Wait.

Once we added packet loss both protocols were almost identical in performance with Go-Back-N sometimes performing slightly worse than Stop-and-Wait. At thirty percent packet loss over a 100MB file size Stop-and-Wait took 516 seconds and Go-Back-N took 559 seconds. At ten percent packet loss for a fifty MB file size Stop-and-Wait took 70.8 seconds and Go-Back-N took 72.4 seconds.

The reason for Go-Back-N's weaker performance when we added packet losses is that testing was completed using localhost, where all packets will realize essentially a round trip time of zero. Typically, Go-Back-N performs better when the communication link is utilized while waiting for packets to traverse the network. With no travel time there is little gain to pipelining. When packets are lost, Go-Back-N must resend the entire window and this means that in addition to outcome packets, Go-Back-N is also resending all the packets that Stop-and-Wait will not. If the tests were performed over an actual network with actual round trip time, Go-Back-N would win out over Stop-and-Wait but this was not the case with the localhost.

2. There is a noticeable decrease in protocol performance with increasing data loss due primarily to increases in the time it takes to resend data (e.g., waiting for timeouts) rather than the amount of data being sent (in this case the total file size). For example, when transmitting a 100 MB file using the Stop-and-Wait protocol, the total time of transmission increased dramatically from 1.9 seconds at 0% loss to 147 seconds at 10% loss, 322 seconds at 20% loss, and finally 516 seconds at 30% loss. Corresponding to

these transmission times, the throughput of Stop-and-Wait dropped from approximately 55 MB/second at 0% to only 0.2 MB/second at 30% loss, while retransmitted packets increased from not at all at 0% to approximately 12,000; then 26,000; then 44,000 at 30% loss. The Go-Back-N protocol displayed a very similar trend.

In the case of Selective Repeat, by comparison, the results for the 100 MB transmission with 10% loss resulted in a total time of 75.8 seconds, while 20% yielded a total time of 129.7 seconds, and 30% yielded 194.3 seconds. Thus, in summary, for every level of packet loss, the Selective Repeat protocol was approximately twice as fast as either the Stop-and-Wait or Go-Back-N protocols, yet it maintained 2-3 times the throughput of those other protocols even though they lost nearly the same number of packets. The key distinction is that Selective Repeat retransmits only packets that were lost, whereas Stop-and-Wait and Go-Back-N retransmit packets that have already been successfully received.

3. The three protocols exemplify the trade-off between simplicity and efficiency in different ways. Stop-and-Wait is the simplest and easiest to develop and to debug; it only allows for one packet to be outstanding at any time, has no windowing or buffering requirements, and is very low in memory usage. However, it is not very efficient if long round trip times occur. Go-Back-N is more complex because of the need for a sliding window, base/next sequence tracking, a retransmission timer, and cumulative ACKs. It is more efficient at keeping the link utilized in situations where there is little or no loss; however, there is a large amount of wasted bandwidth in situations of packet loss because of the need to resend an entire window of data. Selective Repeat is the most complex protocol of the three because both sender and receiver must track individual ACKs per packet sent and both sender and receiver must buffer and reorder packets that arrive out of order. However, Selective Repeat has the best utilization of bandwidth in the presence of packet loss, as it will only resend packets that are missing.
4.
 - Low-Latency Links: When you are working on a local network or a short WAN connection and the Round Trip Time is near zero, pipelining really doesn't give you that much advantage. Our own testing shows that using Stop-and-Wait is just as good for preventing data loss as Go-Back-N does when a loss occurs; therefore, the simplicity of Stop-and-Wait has no real economic benefit in this scenario.
 - Small/Cheap Devices: Many small, inexpensive IoT devices, such as basic temperature sensors, lack sufficient storage/processing capabilities to accommodate window buffers; therefore, Stop-and-Wait Protocol is the only viable option for these devices.
 - High-Loss Links: The cost of a packet drop with STOP-and-WAIT is only the single packet you need to resend; With a Window Protocols, if there are multiple packet drops, you will have multiple packets to resend. Since every transmitted packet contains new sequence numbers, there's an additional cost due to wasted/bad data resends when you use window protocols on unreliable communications links.

- Simple Requests and Responses: When sending simple messages where only one message will ever be outstanding and nothing is gained by using a full window, stop-and-wait is really all you need.

Conclusion

According to our test data, different networks perform better with different protocols. When the network is clean or when round trip time is long (RTT is large), Go-Back-N's pipelining will yield a significant increase in speed relative to Stop-and-Wait as it fully utilizes the available bandwidth, sending multiple files across the same path in very short time intervals. Again, our data demonstrates that once there are dropped packets (due to heavy traffic) and round trip time approaches zero, Go-Back-N's practice of retransmitting entire windows eliminates this speed advantage making both protocols equal at transmitting files. In other words, the performance of Stop-and-Wait is very independent of congestion. This was true when we tested both protocols under heavy loss conditions because of the conditions created by having one packet in flight compared to the many packets sent by Go-Back-N during that time period (elastic stretch).

This is really exciting to highlight our clear winner for the selected option we created during our bonus challenge, which was Selective Repeat. It finished the transfer of 100 MB of data with 30% packet loss in just 194 seconds, while transferring the same amount of data using the other two protocols would have taken approximately 516 – 559 seconds; thus, Selective Repeat enabled the transfer of the data less than half the time than the other two options. We could easily build on Selective Repeat if we wanted to maximize throughput. One way would be to create an adaptive window mechanism that would grow when the channel is clear and shrink when the channel is congested, which would allow the sender to better match their sending rate with the receiver's receiving capacity. We also could replace the fixed timeout period with one that adjusts to measured round trip time, similar to TCP's implementation of this, which would reduce both early and late retransmissions.